ATM

Acornacademy Java Project

팀장 황스일
팀원 이현겸
팀원 손영석
팀원 장해든

목차

- 개요
- 프로젝트 구조
- 주요 기능
- 시연
- 후기

프로젝트 개요

ATM 관리 프로그램을 만들며 그동안 배운 것을 활용해 본다.

프로젝트 개요

역할 분담

사용자 인증 및 보안 / 계좌, 고객 데이터 관리 → **이현겸**

입출금, 계좌 이체 및 잔액 제어

거래 내역 기록(로그)

→ **손영석**

기기 관리자 기능 → **장해든**

프로젝트 개요

역할 분담

이현겸

손영석

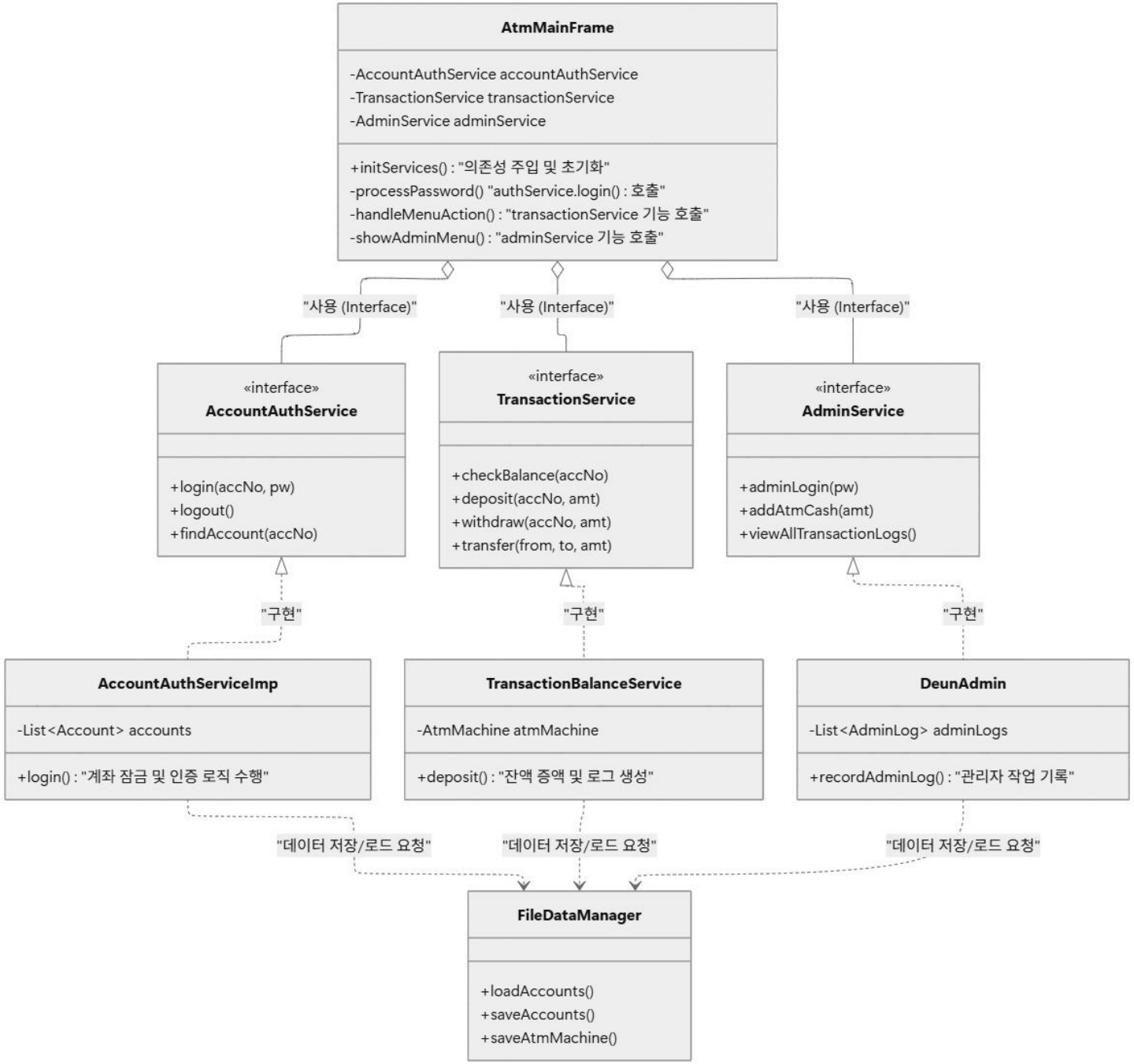
장해든

기능 취합 및 프론트 구현

황스일

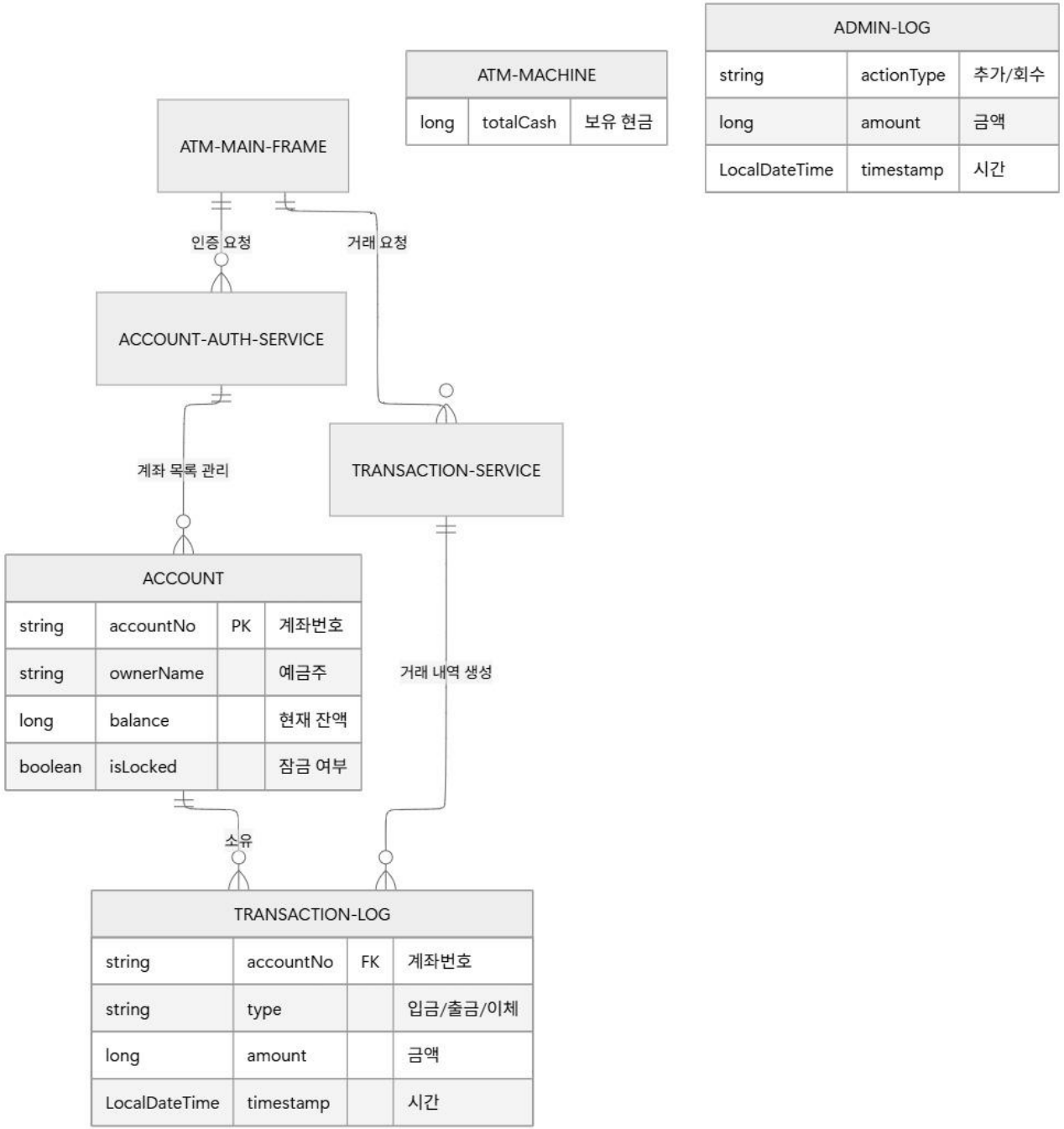
프로젝트 구조

시스템 구조



프로젝트 구조

데이터 구조 (ERD)



주요 기능 – 엔티티

Account.java

```
public class Account {
    // 계좌번호
    private String accountNo;
    // 예금주명
    private String ownerName;
    // 비밀번호
    private String password;
    // 현재 잔액
    private long balance;
    // 비밀번호 오류 횟수
    private int passwordErrorCount;
    // 계좌 잠금 상태
    private boolean isLocked;
    // 계좌별 거래 내역
    private final List<TransactionLog> accountRecord = new ArrayList<>();
    // 로그인 상태
    private boolean isLogined;

    // 계좌 생성자
```

계좌 정보 객체를 생성하기 위한 Account **클래스이다.**

주요 기능 – 엔티티

AtmMachine.java

```
public class AtmMachine {
    private long totalCash;
```

Atm 기기 객체를 생성하기 위한 AtmMachine 클래스이다.

주요 기능 – 엔티티

AdminLog.java

```java
public class AdminLog {
    private String actionType;
    // 관리자 작업 유형 (예: "현금추가", "현금회수")
    private long amount;
    // 처리 금액
    private LocalDateTime timestamp;
    // 작업 시간
```

관리자의 행동을 추적할 수 있게 하는 로그 객체를 생성하기 위한 AdminLog 클래스이다.

주요 기능 – 엔티티

TransactionLog.java

```java
public class TransactionLog {
    // 거래가 발생한 계좌번호
    private final String accountNo;
    // 거래 유형 (입금, 출금, 이체 등)
    private final String transactionType;
    // 거래 금액
    private final long amount;
    // 거래가 끝난 직후 계좌 잔액
    private final long balanceAfter;
    // 거래 발생 시각
    private final LocalDateTime timestamp;
```

거래를 추적할 수 있는 로그 객체를 생성하기 위한 클래스이다.

주요 기능 – 사용자 관리

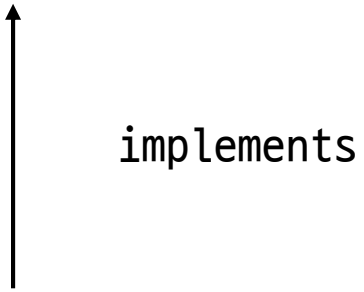
구조

엔티티

Account.java

구현

AccountAuthService.java



AccountAuthServiceImp.java

주요 기능 – 사용자 관리

구현 기능

- 초기 데이터 세팅
- 로그인 및 로그 아웃
- 비밀번호 오류 횟수 카운트 및 계좌 잠금

주요 기능 – 사용자 관리

AccountAuthService.java (인터페이스)

```java
public interface AccountAuthService {

    void initSampleAccounts(); // 초기 데이터 세팅 (샘플 계좌 3~5개 생성
    Account findAccount(String accountNo); // 계좌번호로 특정 계좌 객체 찾아오기
    List<Account> getAccounts(); // 전체 계좌 목록 가져오기 (UI 연동용)

    // 사용자 인증 및 보안
    boolean login(String accountNo, String password); // 로그인 체크
    void logout(); // 로그아웃 및 세션 종료
    void handlePasswordError(Account account); // 비밀번호 오류 횟수 카운트 및 계좌 잠금
```

사용자 인증, 계좌 관리 기능 중심으로 묶은 인터페이스이다.

주요 기능 – 사용자 관리

AccountAuthServiceImp.java

```java
public class AccountAuthServiceImp implements
AccountAuthService {

    private List<Account> accounts;
    private final FileDataManager fileDataManager;
```

AccountAuthService 인터페이스를 상속 받아 실제 메서드를 구현하는 서비스 클래스이다.
생성된 계좌 객체를 ArrayList 타입으로 저장한다.

Json 파일에 데이터를 저장하기 위해 FileDataManager를 사용한다.

주요 기능 – 사용자 관리

AccountAuthServiceImp.java – AccountAuthServiceImp() (생성자)

```java
public AccountAuthServiceImp(FileDataManager fileDataManager) {
    this.fileDataManager = fileDataManager;
    this.accounts = fileDataManager.loadAccounts();

    if (this.accounts.isEmpty()) {
        initSampleAccounts();
    } else {
        for (Account account : this.accounts) {
            account.setLogined(false);
        }
    }
}
```

AccountAuthServiceImp 객체를 생성하며, 파일에서 불러온 계좌가 없다면 기본 상태로 초기화 하는 메서드이다.
프로그램 시작 시 모든 계좌를 강제로 로그아웃해주어 예외 상황을 예방한다.

주요 기능 – 사용자 관리

AccountAuthServiceImp.java - initSampleAccounts()

```java
@Override
public void initSampleAccounts() {
    // 이 메서드는 이제 파일이 없을 때 최초 데이터를 생성하는 역할
    this.accounts = new ArrayList<>();
    accounts.add(new Account("000001", "홍길동", "1234", 1900000));
    accounts.add(new Account("000002", "김철수", "1234", 656000));
    accounts.add(new Account("000003", "박영희", "1234", 45000));
    accounts.add(new Account("000004", "김덕배", "1234", 2187600));
    fileDataManager.saveAccounts(accounts); // 생성된 초기 데이터를 파일에 저장
}
```

Json 파일이 존재하지 않을 때 초기 데이터와 파일을 생성하는 역할이다.
계좌번호, 이름, 비밀번호, 금액을 입력한다.

주요 기능 – 사용자 관리

AccountAuthServiceImp.java - initSampleAccounts()

```java
@Override
public Account findAccount(String accountNo) {
    for (Account account : accounts) {
        if (account.getAccountNo().equals(accountNo)) {
            return account;
        }
    }
    return null;
}
```

계좌번호를 받아 해당 계좌번호를 가지고 있는 계좌 객체를 반환해 주는 메서드이다.

주요 기능 – 사용자 관리

AccountAuthServiceImp.java - getAccounts()

```java
@Override
public List<Account> getAccounts() {
    return this.accounts;
}
```

UI에서 전체 계좌 목록을 표시하기 위해 리스트를 반환하는 메서드이다.

주요 기능 – 사용자 관리

AccountAuthServiceImp.java - login()

계좌번호와 비밀번호를 입력 받는다.
로그인 상태(중복 로그인)인지 확인하고,
모든 검사를 통과하면 비밀번호 오류 횟수를 초기화 하고 로그인 플래그를 true로 바꾸는 메서드이다.

```java
@Override
public boolean login(String accountNo, String password) {
    Account account = findAccount(accountNo);

    // 중복 로그인 방지
    for(Account a : accounts){
        if(a.isLogined() == true){
            System.out.println("로그아웃이 필요합니다.");
            return false;
        }
    }

    if (account == null) {
        System.out.println("계좌가 존재하지 않습니다.");
        return false;
    }

    if (account.isLocked()) {
        System.out.println("계좌가 잠겨있습니다. 해제하려면 은행을 방문해 주세요.");
        return false;
    }

    if (!account.getPassword().equals(password)) {
        System.out.println("비밀번호가 일치하지 않습니다.");
        handlePasswordError(account);
        return false;
    }

    System.out.println("로그인 성공");
    account.resetPasswordErrorCount();
    account.setLogined(true);
    return true;
}
```

주요 기능 – 사용자 관리

AccountAuthServiceImp.java - logout()

```java
@Override
public void logout() {
    for(Account account : accounts) {
        if (account.isLogined() == true) {
            System.out.println("로그아웃 하였습니다.");
            account.setLogined(false);
        }
    }
}
```

계좌 객체 리스트 중 로그인 된 계정을 찾아 로그인 플래그를 false로 만드는 메서드이다.

주요 기능 – 사용자 관리

AccountAuthServiceImp.java - handlePasswordError()

계좌 객체마다 비밀번호 오류 횟수를 카운팅 하고, 세 번 틀렸을 시 계좌를 잠그는 메서드이다.

```
@Override
public void handlePasswordError(Account account) {

    account.increasePasswordErrorCount();
    System.out.println("비밀번호 오류 횟수: " + account.getPasswordErrorCount());
    if(account.getPasswordErrorCount() >= 3){
        account.setLocked(true);
        System.out.println("비밀번호 오류로 인해 계좌가 잠금되었습니다. 해제하려면 은행을 방문해 주세요.");
    }

}
```

주요 기능 – 거래 및 입출금

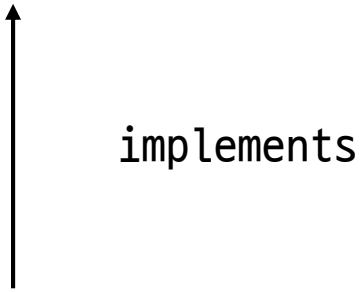
구조

엔티티

TransactionLog.java

구현

TransactionService.java



TransactionBalanceService.java

주요 기능 – 거래 및 입출금

구현 기능

- 입출금 및 잔액 제어
- 계좌 이체
- 거래 내역 기록(Log)

주요 기능 – 거래 및 입출금

TransactionService.java (인터페이스)

```
public interface TransactionService {
    // 입출금 및 잔액 제어
    long checkBalance(String accountNo); // 현재 잔액 확인
    void deposit(String accountNo, long amount); // 현금 입금
    void withdraw(String accountNo, long amount) throws Exception; // 현금 출금 (잔액 차감 및 단위 제한,
부족 시 제한)

    // 계좌 이체
    void transfer(String fromAccountNo, String toAccountNo, long amount) throws Exception;
    // 보내는 계좌 차감 + 받는 계좌 합산, 없는 계좌 체크

    // 거래 내역 기록 (Log)
    void recordTransaction(String accountNo, String type, long amount); // 입금/출금/이체 발생 시 시간,
금액, 거래 유형 기록
    List<String> getRecentHistory(String accountNo); // 최근 거래 내역 리스트 보기
}
```

거래 및 입출금, 추적을 위한 로그 작성 기능 구현을 위한 인터페이스를 생성한다.

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – **클래스 내부 필드**

```
private final List<Account> accountList;
private AtmMachine atmMachine;
private final FileDataManager fileDataManager;

private static final DateTimeFormatter HISTORY_TIME_FORMAT =
DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
```

계좌 목록 List<Account> => ATM 거래 시에 atm 내부에 현금 잔액을 변경위해
Atm 기계 AtmMachine atmMachine => atm 기계에 잔액을 변경하기 위해
파일(.json) 저장 FileDataManager fileDataManager => 전체 거래 기록 .json 로 저장
거래 시각 포맷 DateTimeFormatter HISTORY TIME FORMAT => 시간을 사용자가 읽기 쉬운 포맷으로 변경위해

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – TransactionBalanceService() (생성자)

```
public TransactionBalanceService(List<Account> accountList,
AtmMachine atmMachine, FileDataManager fileDataManager) {
    this.accountList = accountList;
    this.atmMachine = atmMachine;
    this.fileDataManager = fileDataManager;
}
```

거래 시각을 제외한 나머지 필드는 생성자로 받는다.

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – findAccount()

```java
private Account findAccount(String accountNo) {
    for (Account account : accountList) {
        if (account.getAccountNo().equals(accountNo)) {
            return account;
        }
    }
    return null;
}
```

계좌번호를 매개변수로 받아 계좌를 Account 타입으로 반환해주는 메서드
=> 계좌 번호로 계좌를 조회

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – validateBalance()

```java
private void validateBalance(Account account)
throws Exception {
    if (account.getBalance() < 0) {
        throw new Exception("잔액이 부족합니다.");
    }
}
```

계좌(객체)를 매개변수로 받아 계좌를 잔액이 0보다 작을 경우에 잔액 검증을 해주는 메서드
=> 잔액 부족하면 거래 중지를 위한 메서드이다.

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – recordTransaction()

```java
private void recordTransaction(Account account, String type, long amount) {
    TransactionLog transactionLog = new TransactionLog(
            account.getAccountNo(),
            type,
            amount,
            account.getBalance()
    );
    account.addTransactionLog(transactionLog);
}
```

계좌, type(입/출금/이체정보) , 거래량을 가지고 새로운 거래정보 객체를 만들어서 계좌 객체 안에 있는 거래 내역에 추가한다.

=> 요약 : 계좌 정보를 계좌 내부의 거래 내역에 추가

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – formatTransactionType()

```java
public String formatTransactionType(String type) {
    if (type.equals("recordDeposit")) {
        return "입금";
    } else if (type.equals("recordWithdraw")) {
        return "출금";
    } else if (type.equals("recordTransferPlus")) {
        return "이체받음";
    } else if (type.equals("recordTransferMinus")) {
        return "이체보냄";
    }
    return type;
}
```

거래 종류(type)를 매개변수로 받아서

거래 종류에 맞는 각각의 사용자가 알아보기 쉬운 문자열로 반환해주는 메서드이다.

=> 거래종류(type) 로그를 작성할 때 사용. 지금은 public으로 만들어졌지만 private 으로 바꿔도 무관하다.

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – formatHistory()

```java
public String formatHistory(TransactionLog transactionLog) {
    String amountPrefix =
            (transactionLog.getTransactionType().equals("recordWithdraw")
                    || transactionLog.getTransactionType().equals("recordTransferMinus")) ? "-" : "+";

    return transactionLog.getTimestamp().format(HISTORY_TIME_FORMAT)
            + " | "
            + formatTransactionType(transactionLog.getTransactionType())
            + " | "
            + amountPrefix + transactionLog.getAmount()
            + " | 잔액: "
            + transactionLog.getBalanceAfter();
}
```

계좌 거래 기록을 가지고 거래종류(type)에 따라
사용자가 알아보기 쉽게 거래 기록을 만들어주는 메서드

=> 요약 : 계좌 기록(객체)를 사용자가 편의성을 위해 계좌 기록
(문자열)로 만듦

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – checkBalance()

```java
@Override
public long checkBalance(String accountNo) {
    Account account = findAccount(accountNo);
    return account.getBalance();
}
```

계좌 번호를 받아 내장 메서드로 계좌 객체를 가져오고 그 계좌의 잔액을 반환하는 메서드

요약 => 계좌번호로 계좌 잔액을 조회한다.

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – deposit()

```java
@Override
public void deposit(String accountNo, long amount) {
    Account account = findAccount(accountNo);
    if (amount % 1000 == 0) {
        account.setBalance(account.getBalance() + amount);
        atmMachine.addCash(amount);
        recordTransaction(account, "recordDeposit", amount);
        fileDataManager.saveAccounts(accountList);
        fileDataManager.saveAtmMachine(atmMachine);
    }
}
```

계좌번호, 그리고 거래금액을 가져와 1000 원 이상 단위인지 확인하고
Atm 기계 잔액, 계좌 잔액을 증가시킨 뒤 해당 기록을 저장하는 메서드이다.

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – withdraw()

```java
@Override
public void withdraw(String accountNo, long amount) throws
Exception {
    Account account = findAccount(accountNo);
    if (amount % 10000 == 0) {
        account.setBalance(account.getBalance() - amount);
        validateBalance(account);
        atmMachine.withdrawCash(amount);
        recordTransaction(account, "recordWithdraw", amount);
        fileDataManager.saveAccounts(accountList);
        fileDataManager.saveAtmMachine(atmMachine);
    }
}
```

계좌번호, 그리고 거래금액을 가져와서 메서드를 호출한 쪽으로 예외를 넘기고 10000원 이상의 단위 출금인지 확인한 뒤 유효한 계좌인지 검증(출금액 보다 잔액이 많은지 확인) 후 Atm 기계 잔액, 계좌 잔액 감소

거래기록을 남기고, 저장하는 메서드이다.

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – transfer()

```
@Override
public void transfer(String fromAccountNo, String toAccountNo, long
amount) throws Exception {
    Account fromAccount = findAccount(fromAccountNo);
    Account toAccount = findAccount(toAccountNo);

    fromAccount.setBalance(fromAccount.getBalance() - amount);
    validateBalance(fromAccount);
    toAccount.setBalance(toAccount.getBalance() + amount);

    recordTransaction(fromAccount, "recordTransferMinus", amount);
    recordTransaction(toAccount, "recordTransferPlus", amount);
    fileDataManager.saveAccounts(accountList);
}
```

계좌 이체 보낼 계좌번호, 받을 계좌번호 그리고 거래금액을 가져와서 메서드를 호출한 쪽으로 예외를 넘기고,
계좌 번호로 계좌를 찾아 유효한 계좌인지 검증(보낸 금액 보다 잔액이 많은지 확인)

거래기록을 남기고, 저장하는 메서드

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – recordTransaction()

```java
@Override
public void recordTransaction(String accountNo,
String type, long amount) {
    Account account = findAccount(accountNo);
    recordTransaction(account, type, amount);
}
```

계좌번호와, 거래량, 거래종류(입/출금/계좌이체)를
가져와서 거래 기록을 남긴다.

Type 으로는
입금(recordDeposit),
출금(recordWithdraw),
계좌이체 보냄(recordTrnasferMinus),
계좌이체 받음(recordTransferPlus)이 있다.

주요 기능 – 거래 및 입출금

TransactionBalanceService.java – getRecentHistory()

```java
@Override
public List<String> getRecentHistory(String accountNo) {
    Account account = findAccount(accountNo);
    List<String> recentHistory = new ArrayList<>();

    if (account == null) {
        return recentHistory;
    }

    List<TransactionLog> accountRecord = account.getAccountRecord();

    // 최신 기록이 먼저 오도록 뒤에서부터 역순으로 반환용 리스트 생성
    for (int i = accountRecord.size() - 1; i >= 0; i--) {
        recentHistory.add(formatHistory(accountRecord.get(i)));
    }

    return recentHistory;
}
```

계좌번호를 가져와서 계좌를 조회하고.
새로운 List<String> recentHistory을 만들어서
이 recentHistory이 null이 아닌지 확인하고.
계좌의 기록을 List<TransationLog> 형식으로 가져온다.

그리고 가져온 계좌 기록을 역순으로 순회하며
사용자 편의성에 맞는 포맷 시각으로 recentHistroy List에 추가한다.

그리고 recentHistory을 반환한다.

=> 계좌 기록을 가져와 역순으로 만들고 저장후 반환.

주요 기능 – ATM 관리자 기능

구조

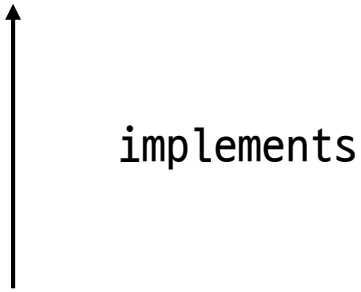
엔티티

AdminLog.java

AtmMachine.java

구현

AdminService.java



DeunAdmin.java

주요 기능 – ATM 관리자 기능

구현 기능

- 관리자 전용 암호 인증
- ATM 기기 내 총 현금 잔고 확인
- ATM 기기에 현금 추가 / 빼기 기능
- 현금 추가하고 뺄 경우의 로그 기록
- 거래 내역 확인 - Log에 접근하여 확인

주요 기능 – ATM 관리자 기능

AdminService.java (인터페이스)

```java
public interface AdminService {
    // 기기 관리자 기능
    boolean adminLogin(String adminPassword); // 관리자 전용 암호 인증
    long checkAtmTotalCash(); // ATM 기기 내 총 현금 잔고 확인
    void addAtmCash(long amount); // ATM 기기에 현금 추가하기
    void withdrawAtmCash(long amount); // ATM 기기에 현금 빼기
    void recordAdminLog(String type, long amount, LocalDateTime timestamp);
    // 현금 추가하고 뺄 경우의 로그 기록

    List<String> viewAllTransactionLogs(); // 거래 내역 확인 - Log 에 접근하여 확인
}
```

기기 관리자 기능 (6가지) 구현을 위한 인터페이스를 생성한다.

주요 기능 – ATM 관리자 기능

DeunAdmin.java – DeunAdmin() (생성자)

```java
public DeunAdmin(AtmMachine atmMachine, List<Account> accounts,
TransactionBalanceService transactionBalanceService, FileDataManager fileDataManager) {
    this.atmMachine = atmMachine;
    this.accounts = accounts;
    this.transactionBalanceService = transactionBalanceService;
    this.fileDataManager = fileDataManager;
    this.adminLogs = fileDataManager.loadAdminLogs();
}
```

주요 기능 – ATM 관리자 기능

DeunAdmin.java – checkAtmTotalCash()

```
@Override
public long checkAtmTotalCash() {
    return atmMachine.getTotalCash();
}
```

ATM 객체의 총 잔고를 반환해주는 메서드이다.

주요 기능 – ATM 관리자 기능

DeunAdmin.java – addAtmCash()

```
@Override
public void addAtmCash(long amount) {
     atmMachine.addCash(amount);
     recordAdminLog( "현금추가", amount, LocalDateTime.now());
     System.out.println("현재 ATM 총금액 : " + atmMachine.getTotalCash() + " 원");
}
```

ATM 객체 잔고에 현금을 추가하고, 로그를 생성하는 메서드이다.

주요 기능 – ATM 관리자 기능

DeunAdmin.java – withdrawAtmCash()

```
@Override
public void withdrawAtmCash(long amount) {
    atmMachine.withdrawCash(amount);
    recordAdminLog( "현금회수", amount, LocalDateTime.now());
    System.out.println("현재 ATM 총금액 : " + atmMachine.getTotalCash() + " 원");
}
```

ATM 객체 잔고에 현금을 회수하고. 로그를 생성하는 메서드이다.

주요 기능 – ATM 관리자 기능

DeunAdmin.java – recordAdminLog()

```
@Override
public void recordAdminLog(String type, long amount,
LocalDateTime timestamp) {
     adminLogs.add(new AdminLog(type, amount, timestamp));
}
```

ATM 객체 잔고에 현금을 추가하고 회수한 로그를 모두 생성하는 메서드이다.

주요 기능 – ATM 관리자 기능

DeunAdmin.java – viewAllTransactionLogs()

```java
@Override
public List<String> viewAllTransactionLogs() {
    // 정렬을 위해 시간과 메시지를 묶어주는 내부 클래스
   class LogEntry {
        LocalDateTime time;
        String logMessage;
        LogEntry(LocalDateTime time, String logMessage) {
            this.time = time;
            this.logMessage = logMessage;
        }
    }
```

거래 내역의 거래 시간, 거래 메시지 (내용)를보여주는 메서드이다.

```java
List<LogEntry> entries = new ArrayList<>();

    // 모든 계좌를 순회하며 거래 내역 수집
   for (Account account : this.accounts) {
        for (TransactionLog log : account.getAccountRecord()) {
            // 기존 서비스의 포맷 기능을 사용하여 로그 문자열 생성
        String formattedLog = transactionBalanceService.formatHistory(log);
            // 관리자용으로 [이름(계좌번호)] 정보를 앞에 추가
        String adminViewLog = String.format("[%s(%s)] %s", account.getOwnerName(), account.getAccountNo(), formattedLog);
            entries.add(new LogEntry(log.getTimestamp(), adminViewLog));
        }
    }

    // 모든 거래 내역을 시간순(오름차순)으로 정렬
   entries.sort(Comparator.comparing(e -> e.time));

    // 정렬된 결과에서 메시지만 추출하여 리스트로 반환
   List<String> allLogs = new ArrayList<>();
    for (LogEntry e : entries){
        allLogs.add(e.logMessage);
    }
    return allLogs;
}
```

주요 기능 – ATM 관리자 기능

DeunAdmin.java – adminLogin()

```
@Override
public boolean adminLogin(String adminPassword)
{   // 관리자 로그인
    // 비번은 1234 임
   String password = "1234";
    if (adminPassword.equals(password)){
        return true;
    } else {
        return false;
    }
}
```

관리자 전용 모드로 접근하려면 관리자 로그인을 해야 한다.

adminPassword의 값을 .equals 를 통해 (password)의 값과 동일한지 확인한다.

Boolean 을 통해 true / false 를 통해 로그인 성공 / 실패 여부를 판단한다.

시영

후기 – 객체지향적 설계

추상화

- 공통 기능을 인터페이스로 분리하고, 이를 구현 클래스에서 구현했다.
- 거래 기능이 어떤 동작을 해야 하는지와 그 동작을 실제로 어떻게 처리 해야 하는지를 분리했다.

후기 – 객체지향적 설계

책임 분리

하나의 클래스가 모든 책임을 짊어지지 않고

계좌 상태는 Account ,
Atm 현금 상태는 AtmMachine,
거래 기록은 TransationLog,
파일 저장은 FileDataManager
거래 조정은 TransactionBalacneService
처럼 책임을 분산시켜 단일 책임 원칙을 준수한다.

후기 – 객체지향적 설계

캡슐화

계좌 찾기, 유효한 잔액 검증, 거래 기록 남기기 등의 메서드를 private 접근 제한자로 감싸 중복 행위를 메서드로 분리함으로써 캡슐화 되었다.

후기 – 개인별 후기

황스일	이번 프로젝트를 통해 팀원과 협업 하고 코드를 취합하여 프로그램을 완성시키는 과정을 통해 많이 성장했다고 느꼈다.
이현겸	아무리 코딩을 잘 해봐야 객체지향적으로 사고하지 않으면 자바는 어려울 수 있겠다는 생각이 들었고. 재미있는 팀프로젝트였던 것 같다.
손영석	프로그래밍 언어를 오랜만에 배우다 보니. 처음에는 배운 내용만으로 무언가를 만들 수 있을지 의문이 들었다. 하지만 기존에 학습한 내용을 다시 복습하고 공부하면서. 프로젝트를 시작한 이후 자바를 더 깊이 이해할 수 있게 된 것 같다. 또한 객체지향적인 코딩을 위해 노력했지만. 구현 메서드 안에서 반복되는 검증과 예외 처리 로직을 별도의 메서드로 분리하는 과정은 쉽지 않았다. 그럼에도 불구하고 배운 내용을 실제로 적용해볼 수 있어서 뿌듯했다.
장해든	자바를 처음 배우면서 진행한 첫 팀 프로젝트라서 많이 어렵고 낯설었다. 하지만 팀원들과 함께 헷갈리거나 모르는 부분을 해결해 나가고, 직접 실행해 보면서 내가 구현하고자 한 기능들을 하나하나 완성해 나갈 수 있었다. 그 결과 최종적으로 프로젝트를 마무리할 수 있어 뿌듯했고, 스스로 더 발전하고 성장할 수 있는 시간이었다고 느껴 뜻깊은 경험이었다. 클래스, 객체, 생성자와 같은 개념들을 수업 시간에 배우고 간단한 실습만 하다가, 이를 조금 더 큰 규모의 프로젝트에 적용하려고 하니 익숙하지 않아서 어디에 무엇을 만들어야 하는지 자주 헷갈렸다. 또한 기능 하나를 구현하는 데에도 생각보다 많은 구조가 필요하다는 것을 알게 되었다. 특히 ATM 기능을 구현하면서 단순히 코드를 작성하는 것이 아니라, 객체를 어떻게 나누고 서로 연결해야 하는지가 중요하다는 점을 배울 수 있었다. 처음에는 많이 막히고 실수도 있었지만, 하나씩 직접 구현해 보면서 자바와 객체지향 구조에 대해 조금 더 이해할 수 있었던 의미 있는 경험이었다.

감사합니다